

React Hooks introduction handlers

1. Definition

React Hooks are functions introduced in React 16.8 that allow developers to use state and lifecycle features in functional components without writing class components.

Hooks make React code simpler, reusable, and easier to manage.

2. Need for Hooks

- **Earlier, state and lifecycle methods were only available in class components.**
 - **Functional components were used only for UI rendering.**
 - **Hooks allow functional components to manage state, side effects, and logic.**
-

3. Important Features of Hooks

- **Use state inside functional components.**
 - **Access lifecycle methods without class components.**
 - **Help in code reuse.**
 - **Reduce complexity of large components.**
-

4. Commonly Used Hooks

1. **useState() – Manages state in functional components.**
 2. **useEffect() – Handles side effects such as API calls and lifecycle operations.**
 3. **useContext() – Accesses global data from context.**
 4. **useRef() – Accesses DOM elements directly.**
-

5. Basic Implementation Example

```
import React, { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>  );  }
export default Counter;
```

Explanation

- **useState(0) creates a state variable count.**
- **setCount() updates the state.**
- **The component re-renders when state changes.**

Definition

useState is a React Hook that allows functional components to create and manage state variables. It was introduced in React 16.8.

Syntax

const [state, setState] = useState(initialValue);

Explanation

- **state** → Current value of the state.
- **setState** → Function used to update the state.
- **initialValue** → Initial value assigned to the state.

Example

```
import React, { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div> ); }
```

Example Explanation

- **count** stores the state value.
- **setCount()** updates the state when the button is clicked.

Real-life Example

A counter application where the number increases when the user clicks a button.

useState with Previous State

Definition

Sometimes a new state depends on the previous state value. In such cases React provides a callback function inside **setState**.

Syntax

setState(previousState => newState);

Example

```
import React, { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  function increase() {
    setCount(prevCount => prevCount + 1); }
  return (
    <div>
      <h2>{count}</h2>
      <button onClick={increase}>Increase</button>
    </div> ); }
```

Explanation

- **prevCount** represents the previous value of state.
- React calculates the new value based on the previous value.

Example

In a like button system, the new number of likes is calculated from the previous likes count.

Definition

useState can also store objects as state. Objects are useful when we need to store multiple related values.

Example

```
import React, { useState } from "react";
function User() {
  const [user, setUser] = useState({
    name: "Rahul",
    age: 22  });
  function updateAge() {
    setUser({
      ...user,
      age: 23  });  }
  return (
    <div>
      <h2>{user.name}</h2>
      <h3>{user.age}</h3>
      <button onClick={updateAge}>Update Age</button>
    </div>  );  }
```

Explanation

- **State contains an object with multiple properties.**
- **The spread operator (...) keeps existing values while updating one property.**

Example

A user profile form storing name, age, and email as an object.

useState with Array

Definition

useState can also manage arrays as state. This is useful when working with lists of data.

Example

```
import React, { useState } from "react";
function Items() {
  const [items, setItems] = useState([]);
  function addItem() {
    setItems([...items, "New Item"]);  }
  return (
    <div>
      <button onClick={addItem}>Add Item</button>
      <ul>
        {items.map((item, index) => (
          <li key={index}>{item}</li>      ))}
      </ul>
    </div>  );  }
```

Explanation

- **items stores an array of elements.**
- **setItems([...items, "New Item"]) adds a new item to the array.**
- **map() is used to display list items.**

Example

A to-do list application where tasks are stored and displayed as an array.

Definition

useEffect is a React Hook used to handle side effects in functional components.

Side effects include API calls, DOM updates, timers, and data fetching.

Syntax

```
useEffect(() => {  
  // side effect code  
});
```

Explanation

- **It runs after the component renders.**
- **It replaces lifecycle methods like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`.**

Example

```
import React, { useEffect } from "react";  
function Example() {  
  useEffect(() => {  
    console.log("Component Rendered");  });  
  return <h2>Hello React</h2>;  }  
}
```

Example Explanation

When the component renders, the message "Component Rendered" is printed in the console.

Real-life Example

Fetching user data from an API when the page loads.

useEffect After Render**Definition**

By default, `useEffect` runs after every render of the component.

Example

```
import React, { useState, useEffect } from "react";  
function Counter() {  
  const [count, setCount] = useState(0);  
  useEffect(() => {  
    console.log("Component rendered");  });  
  return (  
    <div>  
      <h2>{count}</h2>  
      <button onClick={() => setCount(count + 1)}>Increase</button>  
    </div>  );  
}
```

Explanation

- **When the component first loads, `useEffect` runs.**
- **Every time `count` changes, the component re-renders and `useEffect` runs again.**

Example

Updating document title whenever the component renders.

Definition

useEffect can run only when specific state or props change using the dependency array.

Syntax

```
useEffect(() => {  
  // effect  
}, [dependency]);
```

Example

```
import React, { useState, useEffect } from "react";  
function Counter() {  
  const [count, setCount] = useState(0);  
  useEffect(() => {  
    console.log("Count changed");  
  }, [count]);  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      {count}  
    </button>  );  
}
```

Explanation

- The effect runs only when count changes.

Example

Running an API request only when the search value changes.

Run Effects Only Once

Definition

To run **useEffect** only once when the component mounts, an empty dependency array is used.

Syntax

```
useEffect(() => {  
  // effect code  
}, []);
```

Example

```
import React, { useEffect } from "react";  
function App() {  
  useEffect(() => {  
    console.log("Component Mounted");  
  }, [ ]);  
  return <h2>Welcome</h2>;  
}
```

Explanation

- The effect runs only once when the component loads.
- It behaves like `componentDidMount` in class components.

Example

Loading data from a server when the page first opens.

Definition

A cleanup function is used to remove or stop effects when the component unmounts.

Syntax

```
useEffect(() => {  
  // effect code  
  return () => {  
    // cleanup code  };  });
```

Example

```
import React, { useEffect } from "react";  
function Timer() {  
  useEffect(() => {  
    const timer = setInterval(() => {  
      console.log("Running...");  
    }, 1000);  
    return () => {  
      clearInterval(timer);    };  }, []);  
  return <h2>Timer Running</h2>; }  
}
```

Explanation

- **setInterval starts a timer.**
- **Cleanup function clears the timer when the component unmounts.**

Example

Stopping timers, subscriptions, or event listeners.

useEffect with Incorrect Dependency**Definition**

An incorrect dependency occurs when the dependency array is missing or wrong, causing unexpected behavior.

Example

```
import React, { useState, useEffect } from "react";  
function Example() {  
  const [count, setCount] = useState(0);  
  useEffect(() => {  
    console.log("Count is:", count);  }, []);  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      {count}  
    </button>  ); }  
}
```

Explanation

- **The dependency array is empty ([]).**
- **The effect runs only once, so it does not update when count changes.**

Problem

This may lead to stale data or incorrect results.

Example

If a search value changes but the dependency is not added, the API request will not update properly.

Fetching Data with useEffect

Definition

useEffect Hook is used in React functional components to perform side effects such as fetching data from APIs, updating the DOM, or setting timers.

It is commonly used to retrieve data from external servers after a component renders.

Why use useEffect for Data Fetching

- **API calls are side effects.**
- **React components should fetch data after rendering.**
- **useEffect ensures the API call happens at the correct lifecycle stage.**

Syntax

```
useEffect(() => {  
  // API call  
}, []);
```

Steps for Fetching Data

1. **Import useState and useEffect.**
2. **Create a state variable to store data.**
3. **Use useEffect to call the API.**
4. **Convert response into JSON format.**
5. **Store the data in state using setState.**
6. **Display the data using map().**

Example

```
import React, { useState, useEffect } from "react";  
function Users() {  
  const [users, setUsers] = useState([]);  
  useEffect(() => {  
    fetch("https://jsonplaceholder.typicode.com/users")  
      .then(response => response.json())  
      .then(data => setUsers(data)); }, []);  
  return (  
    <div>  
      <h2>User List</h2>  
      <ul>  
        {users.map(user => (  
          <li key={user.id}>{user.name}</li>      ))}  
      </ul>  
    </div>  ); }  
}
```

Explanation

- **useState([])** creates a state variable to store users.
- **useEffect()** runs once when the component mounts.
- **fetch()** sends a request to the API.
- **The response is converted to JSON and stored in state.**
- **map()** displays the data as a list.

Real-life Example

In an online shopping website, product details are fetched from a server API when the product page loads.

Definition : `useContext` is a React Hook that allows functional components to access data from React Context directly, without passing props through multiple components.

Purpose :

It helps avoid prop drilling, where props must be passed through many intermediate components.

Syntax

```
const value = useContext(ContextName);
```

Example

```
import React, { createContext, useContext } from "react";
const UserContext = createContext();
function Child() {
  const user = useContext(UserContext);
  return <h2>{user}</h2>; }
function App() {
  return (
    <UserContext.Provider value="Rahul">
      <Child />
    </UserContext.Provider> ); }
```

export default App;

Explanation

- `createContext()` creates a context object.
- `Provider` supplies the value to child components.
- `useContext()` reads the value from the context.

Real-life Example : In a login system, user information such as username or theme settings can be shared across multiple components using `useContext`.

Multiple useReducers

Definition

React allows using multiple `useReducer` Hooks in a single component.

Each reducer manages a separate part of the component state.

Purpose : Using multiple reducers helps to separate different state logics, which improves code organization.

Working Concept : Instead of one large reducer handling everything, we create separate reducers for different tasks.

Example:

- One reducer manages counter state.
- Another reducer manages user information.

Small Code Concept

```
const [count, dispatchCount] = useReducer(counterReducer, 0)
const [user, dispatchUser] = useReducer(userReducer, {})
```

Advantages

- Code becomes modular and easier to maintain.
- Different states are independent from each other.
- Improves readability and scalability.

Real-life Example : In a user dashboard application:

- One reducer manages notifications.
- Another reducer manages user profile data.

Definition

useReducer is a React Hook used to manage state in functional components using a reducer function and actions.

It works similar to Redux state management and is useful when state changes depend on different actions.

Syntax

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Components of useReducer

1. **State** – The current value of the state.
2. **Dispatch** – A function used to send actions to the reducer.
3. **Reducer Function** – A function that decides how the state should change based on the action.
4. **Initial State** – The starting value of the state.

Working Process

1. The component sends an action using `dispatch()`.
2. The action is passed to the reducer function.
3. The reducer checks the action type.
4. Based on the action, the reducer returns a new state.
5. React updates the component with the new state.

Example (Simple Counter)

```
import React, {useReducer} from "react";
function reducer(state,action){
  if(action.type==="inc") return state+1;
  if(action.type==="dec") return state-1;
  return state;
}
export default function App(){
  const [count,dispatch]=useReducer(reducer,0);
  return(<div>
    <h2>{count}</h2>
    <button onClick={()=>dispatch({type:"inc"})}>+</button>
  </div>);
}
```

Example

A counter system where clicking the button increases the count

useReducer – Complex State and Action

Definition

useReducer is very useful when a component has complex state with multiple values and multiple actions.

Instead of managing many **useState** variables, **useReducer** manages all related state in one reducer function.

Characteristics of Complex State

- **State contains multiple properties.**
- **Different actions update different parts of the state.**
- **Logic becomes easier to manage in a single reducer function.**

Example Structure of Complex State

```
state = {  
  count: 0,  
  step: 1,  
  totalClicks: 0 }  

```

Actions Example

- **increment** → increase counter
- **decrement** → decrease counter
- **setStep** → change step value

Working Process

1. **The component dispatches different actions.**
2. **The reducer receives the current state and action.**
3. **The reducer decides which state property to update.**
4. **It returns the updated state object.**

Small Code Example

```
dispatch({type: "increment"})  
dispatch({type: "setStep", value: 2})
```

Advantages

- **Keeps complex logic organized in one place.**
- **Easier to manage multiple related state updates.**
- **Improves readability of large components.**

Real-life Example

In a shopping cart system, state may contain:

- **product list**
- **quantity**
- **total price**

useReducer can manage all these updates using different actions.

Conclusion

useReducer is preferred for complex state logic and multiple related updates, making code easier to maintain.

useContext Hook

Definition

useContext is a React Hook used to access data from React Context directly in functional components without passing props through multiple levels.

Purpose

In large React applications, passing props through many components is called prop drilling. **useContext** solves this problem by providing global access to shared data.

Syntax

```
const value = useContext(ContextName);
```

Working Steps

1. Create context using **createContext()**.
2. Provide data using **Context.Provider**.
3. Access data in child components using **useContext()**.

Example (within 10 lines)

```
import React, {createContext, useContext} from "react";
const UserContext=createContext();
function Child(){
  const user=useContext(UserContext);
  return <h3>{user}</h3>; }
export default function App(){
  return <UserContext.Provider value="Rahul"><Child/></UserContext.Provider>; }
```

Example

In a website login system, user data such as username, language, or theme can be accessed by many components.

useReducer Hook

Definition

useReducer is a React Hook used for managing state using a reducer function and actions. It is useful when state updates depend on multiple actions or complex logic.

Syntax

```
const [state,dispatch] = useReducer(reducer,initialState);
```

Working Concept

- State stores current value.
- **dispatch()** sends action to reducer.
- Reducer function updates state based on action type.

Example (within 10 lines)

```
import React,{useReducer} from "react";
function reducer(state,action){
  if(action.type==="inc") return state+1;
  return state; }
export default function App(){
  const [count,dispatch]=useReducer(reducer,0);
  return <button onClick={()=>dispatch({type:"inc"})}>{count}</button>; }
```

Example

A counter application where state changes according to user actions.

Fetching Data with useReducer

Definition

useReducer can be used with **useEffect** to manage API requests and multiple states like loading, success, and error.

Purpose

It helps control data fetching logic in a structured way.

Example

```
import React,{useReducer,useEffect} from "react";
const initial={loading:true,data:[]};
function reducer(state,action){
  if(action.type==="success") return {loading:false,data:action.data};
  return state;
}
function App(){
  const [state,dispatch]=useReducer(reducer,initial);
  useEffect(()=>{fetch("https://jsonplaceholder.typicode.com/users")
    .then(r=>r.json()).then(d=>dispatch({type:"success",data:d})});},[]);
  return <div>{state.loading?"Loading":state.data.length}</div>;
}
```

Example

In an online shopping website, product data is fetched from the server and displayed to users.

useState vs useReducer

Feature	useState	useReducer
Definition	Hook used to manage simple state	Hook used to manage complex state
State Logic	Direct state update using setState()	State updated using reducer function
Complexity	Suitable for simple state	Suitable for complex state logic
Code Structure	Simple and short	More structured
Update Method	Uses setState()	Uses dispatch() with actions
Best Use Case	Input fields, simple counters	Forms, shopping cart, complex state

Example

- **useState** is best for simple values like counters or form inputs.
- **useReducer** is better for large applications with complex state logic.